

Partial Differential Equations

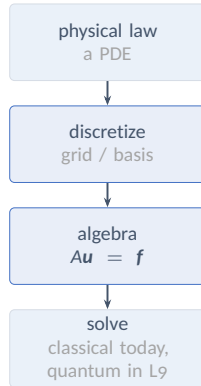
How nature is written down — and how we solve it classically

Lecture 8

October 5, 2026

Agenda

- 1 What a PDE is, and what it describes
- 2 Linear versus nonlinear
- 3 The classical toolbox, and one problem solved by hand



What is a partial differential equation?

- The unknown is a **function** of several variables, e.g. $u(x, t)$ — temperature at position x and time t
- The equation relates its **partial derivatives**: $u_t = \frac{\partial u}{\partial t}$, $u_x = \frac{\partial u}{\partial x}$, $u_{xx} = \frac{\partial^2 u}{\partial x^2}$

Example — the 1D heat equation

$$\underbrace{\frac{\partial u}{\partial t}}_{\text{how fast } u \text{ changes in time}} = \alpha \underbrace{\frac{\partial^2 u}{\partial x^2}}_{\text{how curved } u \text{ is in space}}, \quad \alpha > 0 \text{ (thermal diffusivity)}$$

Vocabulary

- *Order*: highest derivative (heat: 2)
- *ODE*: one variable; *PDE*: two or more

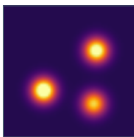
A PDE alone is not a problem

- *Initial condition*: $u(x, 0)$
- *Boundary conditions*: u (or u_x) at the ends

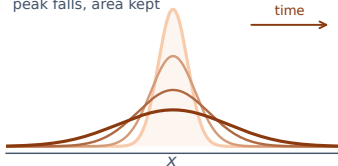
Nature is written in PDEs

Diffusion (parabolic)

$$u_t = \alpha \nabla^2 u$$



peak falls, area kept



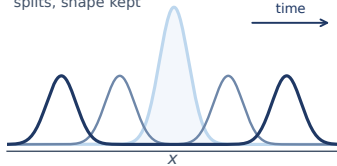
hot plate: spreads and flattens

Waves (hyperbolic)

$$u_{tt} = c^2 \nabla^2 u$$



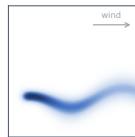
splits, shape kept



pond: travels at speed c

Transport (advection-diffusion)

$$u_t + \mathbf{v} \cdot \nabla u = D \nabla^2 u$$

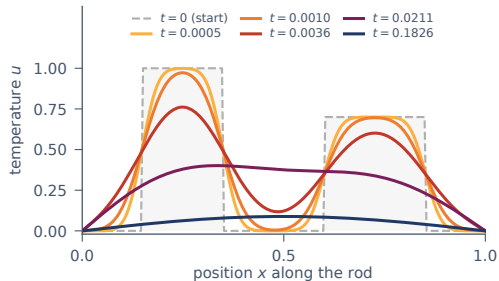


drifts with the flow



smoke: drifts at speed v , spreads

Reading the heat equation



a rod with two hot sections; both ends held at 0°

$u_t = \alpha u_{xx}$ says: **curvature drives change**

- Peak ($u_{xx} < 0$) $\Rightarrow u_t < 0$: it cools
- Valley ($u_{xx} > 0$) $\Rightarrow u_t > 0$: it warms
- Straight line ($u_{xx} = 0$): nothing changes

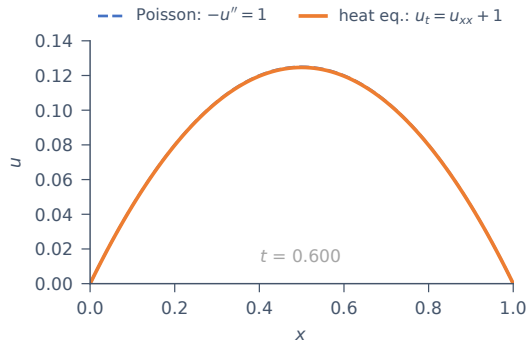
Where it comes from (two physical laws)

- Energy conservation: $\rho c u_t = -q_x$
- Fourier's law: heat flows downhill,
 $q = -k u_x$

$$\Rightarrow u_t = \frac{k}{\rho c} u_{xx} = \alpha u_{xx}$$

Sharp features die fastest: the jagged profile is smooth almost at once, then slowly sags to zero.

Wait long enough: the heat equation becomes Poisson



a wire heated uniformly ($f = 1$), ends cooled to 0

Add a heat source $f(x)$: $u_t = u_{xx} + f$

At steady state nothing changes, $u_t = 0$, so

The running example — 1D Poisson

$$-u''(x) = f(x), \quad x \in (0, 1)$$

$$u(0) = u(1) = 0$$

- No time: a **boundary-value problem**
- Also: electric potential, a loaded string
- For $f = 1$: $u = \frac{1}{2}x(1 - x)$

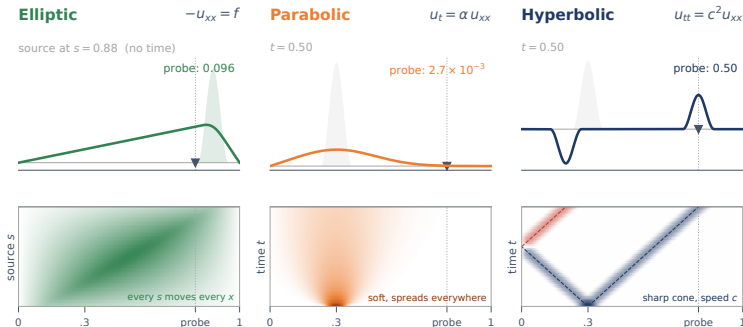
Three families of second-order PDEs

	Elliptic	Parabolic	Hyperbolic
Prototype	Poisson $-\nabla^2 u = f$	Heat $u_t = \alpha \nabla^2 u$	Wave $u_{tt} = c^2 \nabla^2 u$
Describes	equilibrium, steady state	diffusion, smoothing	propagation, vibration
Information	every point affects every point	spreads instantly, decays	travels at finite speed c
Solution	smooth	smooth after $t > 0$	keeps sharp features
Discrete form	one linear system	a linear system <i>per time step</i>	time stepping

- The family is read from the coefficients of $Au_{xx} + Bu_{xy} + Cu_{yy} + \dots$: $B^2 - 4AC < 0, = 0, > 0$
- It tells you which numerical method will work — and which will fail

Takeaway: Poisson (elliptic) is the cleanest test case: no time, one linear system. That is why it is the running example for L8 and L9.

Same poke, three behaviours



Elliptic: no time. Move the source and u changes *everywhere at once*.

Parabolic: the probe is nonzero *immediately*, but tiny. The bump spreads and smooths.

Hyperbolic: the probe reads *exactly* 0 until the pulse arrives ($t \approx 0.43$). Shape is kept.

Information reaches everywhere at once (elliptic), everywhere but damped (parabolic), or only inside a cone of speed c (hyperbolic).

Linear versus nonlinear

Write the PDE as $\mathcal{L}[u] = f$. It is **linear** if the operator satisfies

$$\mathcal{L}[a u + b v] = a \mathcal{L}[u] + b \mathcal{L}[v]$$

— u and its derivatives appear only to the first power, never multiplied together.

Linear

- Heat, wave, Poisson, Laplace
- Schrödinger $i\hbar \psi_t = -\frac{\hbar^2}{2m} \nabla^2 \psi + V\psi$
- Black-Scholes (option pricing)

Superposition: add two solutions, get a solution.
Solve once per mode, sum.

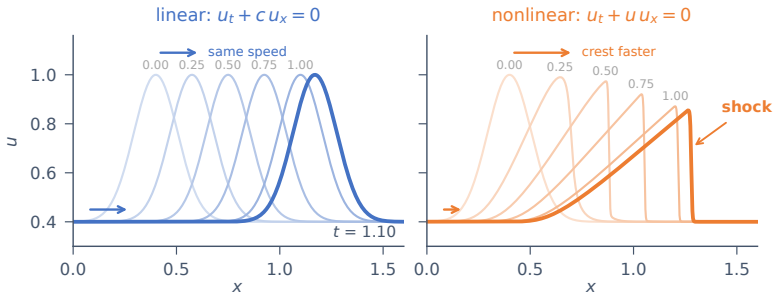
Nonlinear

- Burgers $u_t + u u_x = \nu u_{xx}$
- Navier-Stokes (all of fluid dynamics)
- Allen-Cahn, reaction-diffusion:
 $u_t = \nabla^2 u + u - u^3$

No superposition. Shocks, turbulence, patterns —
and no general solution formula.

■ Is $u_t = x^2 u_{xx}$ linear? (Yes — the coefficient may depend on x ; only u itself may not.)

Nonlinearity in action: a wave that breaks



Linear: every point moves at the same speed c , so the shape is carried unchanged.

Nonlinear: the speed is u , so the crest outruns the base; the front steepens into a **shock** at $t_b = 1 / \max(-u'_0) \approx 0.29$.

Takeaway: Same initial data, one extra factor of u — and a smooth wave becomes a jump. Traffic jams and sonic booms form this way.

Why the distinction matters for computing

Linear PDE

discretize $\implies$ **one linear system**

$$Au = f$$

- Mature, fast solvers
- Matches quantum mechanics, which is itself linear (L9)

Nonlinear PDE

discretize $\implies$ **nonlinear system** $F(u) = 0$

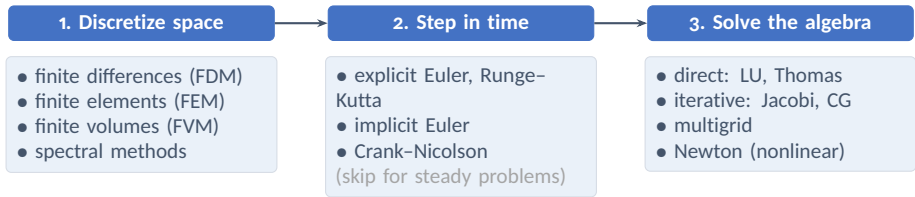
Newton's method: repeat

$$J(u^k) \delta = -F(u^k), \quad u^{k+1} = u^k + \delta$$

- A *sequence* of linear solves
- Needs a good starting guess

Takeaway: Either way, the workhorse is a sparse linear solve. Make that fast and you make almost every PDE solver fast — the promise behind quantum linear solvers.

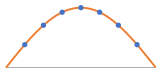
The classical toolbox



This pattern is called the *method of lines*: discretize space first, leaving a large system of ODEs in time.

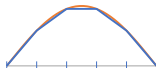
Step 1 turns a function into a vector. Step 3 is where nearly all the computing time goes.

Four ways to turn a function into a vector



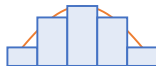
Finite differences

values at grid points; derivatives
from neighbours
simple, regular grids



Finite elements

piecewise polynomials on a
mesh; weak form
complex geometry (structures)



Finite volumes

cell averages; fluxes balanced
across faces
conservation laws (CFD)



Spectral

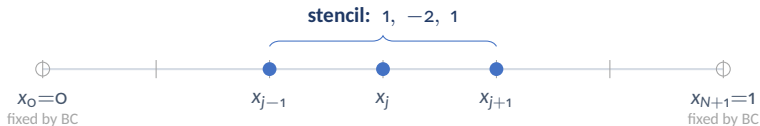
a sum of global basis functions
(Fourier, Chebyshev)
smooth solutions, high accuracy

- All four give **sparse** (FD, FE, FV) or **small dense** (spectral) algebraic systems
- We use finite differences today; **H-DES in L9 is a spectral method** with Chebyshev polynomials

Finite differences

Place N interior points $x_j = jh$, $h = 1/(N + 1)$, and approximate the second derivative by the three-point stencil:

$$u''(x_j) \approx \frac{u_{j-1} - 2u_j + u_{j+1}}{h^2} + \mathcal{O}(h^2).$$



Where it comes from: add the Taylor expansions of $u(x_j \pm h)$; the odd terms cancel.
Halving h quarters the error — and doubles the number of unknowns N .

The PDE becomes a linear system

Write the stencil at every interior point:

$$\underbrace{\frac{1}{h^2} \begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{pmatrix}}_A \underbrace{\begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{N-1} \\ u_N \end{pmatrix}}_u = \underbrace{\begin{pmatrix} f(x_1) \\ f(x_2) \\ \vdots \\ f(x_{N-1}) \\ f(x_N) \end{pmatrix}}_f$$

Properties we will use (here and in L9)

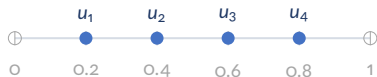
- **Sparse:** 3 nonzeros per row
- **Symmetric positive definite**
- **Ill-conditioned:** $\kappa(A) \approx \frac{4}{\pi^2} (N+1)^2$

Boundary conditions

- Dirichlet $u = g$: known values move to f ($f_1 += g_0/h^2$)
- Neumann $u' = g$: first/last rows of A change

Worked example: a uniformly heated rod

Solve $-u'' = 1$, $u(0) = u(1) = 0$ with $N = 4$ interior points, $h = \frac{1}{5}$.



Multiply through by h^2 : right-hand side $h^2 f = 0.04$

$$\begin{pmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{pmatrix} = \begin{pmatrix} 0.04 \\ 0.04 \\ 0.04 \\ 0.04 \end{pmatrix}$$

What to expect

Exact solution $u(x) = \frac{1}{2}x(1-x)$:

$$u(0.2) = 0.08, \quad u(0.4) = 0.12$$

and symmetric about $x = \frac{1}{2}$.

Gaussian elimination works, but costs $\mathcal{O}(N^3)$ and ignores the zeros.

A tridiagonal matrix deserves a tridiagonal algorithm $\rightarrow$

The Thomas algorithm, by hand*

Gaussian elimination that touches only the three diagonals: $\mathcal{O}(N)$ work.

Forward sweep (eliminate the sub-diagonal)

$$m_i = 2 + c'_{i-1}, \quad c'_i = \frac{-1}{m_i}, \quad r'_i = \frac{0.04 + r'_{i-1}}{m_i}$$

i	m_i	c'_i	r'_i
1	2	$-1/2$	0.02
2	$3/2$	$-2/3$	0.04
3	$4/3$	$-3/4$	0.06
4	$5/4$	—	0.08

Back substitution $u_i = r'_i - c'_i u_{i+1}$

$$u_4 = 0.08$$

$$u_3 = 0.06 + \frac{3}{4}(0.08) = 0.12$$

$$u_2 = 0.04 + \frac{2}{3}(0.12) = 0.12$$

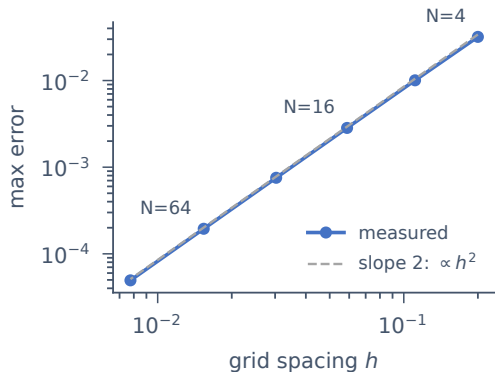
$$u_1 = 0.02 + \frac{1}{2}(0.12) = 0.08$$

Result

$\mathbf{u} = (0.08, 0.12, 0.12, 0.08)$ — **exact** at the nodes

Exact because the stencil has no error for quadratics. Keep these four numbers: in L9 a quantum computer will solve this same system.

Accuracy: the error falls like h^2



A source with a curved answer: $f = \pi^2 \sin(\pi x)$, exact $u = \sin(\pi x)$.

N	h	max error	$\kappa(A)$
4	0.200	3.2×10^{-2}	9.5
16	0.059	2.8×10^{-3}	117
64	0.015	1.9×10^{-4}	1712

- Slope 2 on log-log axes: second-order accurate
- Accuracy costs unknowns, *and* the condition number grows as N^2

Takeaway: Remember $\kappa \sim N^2$. It is harmless for the Thomas algorithm, and it is the term that hurts quantum linear solvers in L9.

Time-dependent problems: one step at a time

Heat equation $u_t = \alpha u_{xx}$: discretize space as before, then step in time with Δt .

Explicit Euler (no linear solve)

$$u_j^{k+1} = u_j^k + r (u_{j-1}^k - 2u_j^k + u_{j+1}^k), \quad r = \frac{\alpha \Delta t}{h^2}$$

Worked, $r = \frac{1}{4}$, ends at 0:

step	(u_1, u_2, u_3, u_4)
0	(0, 100, 100, 0)
1	(25, 75, 75, 25)
2	(31.25, 62.5, 62.5, 31.25)

e.g. $u_1^1 = 0 + \frac{1}{4}(0 - 0 + 100) = 25$

Implicit Euler (one solve per step)

$$(I + rT) \mathbf{u}^{k+1} = \mathbf{u}^k$$

- T is our tridiagonal matrix
- Each step is a Poisson-like system — Thomas again

Crank-Nicolson averages the two: second order in time, and also needs a solve.

What the classical solvers cost

Method	Cost for N unknowns	Notes
Dense LU	$\mathcal{O}(N^3)$	ignores sparsity — never used here
Thomas algorithm	$\mathcal{O}(N)$	1D only (tridiagonal)
Conjugate gradient	$\mathcal{O}(N\sqrt{\kappa})$	SPD matrices; $\kappa \sim N^{2/d}$
Multigrid	$\mathcal{O}(N)$	optimal for elliptic problems in any dimension

So where is the problem? In the size of N itself.

- M points per axis in d dimensions: $N = M^d$ unknowns
- 3D at $M = 1000$: $N = 10^9$ unknowns — per time step
- Parametric sweeps, many coupled fields, or $d \gg 3$ (finance, kinetic theory): out of reach

Takeaway: Classical PDE solvers are already linear in N . A quantum method must beat that, by working with something exponentially smaller than N .

The bridge to L9: N numbers in $\log_2 N$ qubits

An n -qubit state has 2^n amplitudes. Store the solution vector as amplitudes:

$$|u\rangle = \frac{1}{\|u\|} \sum_{j=0}^{N-1} u_j |j\rangle, \quad N = 2^n.$$

Grid points N	Qubits n	Classical memory
4	2	our rod
1,024	10	8 KB
2^{30}	30	8 GB
2^{50}	50	8 PB

Our rod: $u = (0.08, 0.12, 0.12, 0.08) \rightarrow |u\rangle = 0.39 |00\rangle + 0.59 |01\rangle + 0.59 |10\rangle + 0.39 |11\rangle$.

The catch: a quantum state cannot be copied or read out in full. Next lecture: how to *produce* $|u\rangle$ from A and f — and what you can still learn from it.

Summary

- A PDE relates rates of change of an unknown field; with initial and boundary conditions it predicts heat flow, waves, transport
- **Linear** PDEs obey superposition and discretize to one sparse system $A\mathbf{u} = \mathbf{f}$; **nonlinear** ones need Newton's method — many linear solves
- Classical pipeline: discretize space $\rightarrow$ step in time $\rightarrow$ solve the algebra
- Poisson by hand: $\mathbf{u} = (0.08, 0.12, 0.12, 0.08)$ by the Thomas algorithm; error $\mathcal{O}(h^2)$, condition number $\mathcal{O}(N^2)$
- Explicit time stepping is limited by $\alpha\Delta t/h^2 \leq \frac{1}{2}$

Next — L9: solving $A\mathbf{u} = \mathbf{f}$ on a quantum computer: HHL, VQLS, and the ColibriTD H-DES solver.

References

Numerical PDEs

- R. J. LeVeque (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM.
- W. L. Briggs, V. E. Henson, S. F. McCormick (2000). *A Multigrid Tutorial*, 2nd ed. SIAM.
- L. N. Trefethen (2000). *Spectral Methods in MATLAB*. SIAM.
- R. Courant, K. Friedrichs, H. Lewy (1928). "Über die partiellen Differenzengleichungen der mathematischen Physik." *Mathematische Annalen* 100, 32–74.

PDEs and their physics

- L. C. Evans (2010). *Partial Differential Equations*, 2nd ed. AMS Graduate Studies in Mathematics 19.
- G. B. Whitham (1974). *Linear and Nonlinear Waves*. Wiley — Burgers' equation and shock formation.

All animations on these slides are numerical solutions computed for this course (finite differences, Godunov upwinding for Burgers).